

Technical Requirements Document: SUHI Calculation Engine

1. Objective

To build an automated, serverless Python script that calculates the monthly Surface Urban Heat Island (SUHI) effect for municipal wards in Indian cities, starting from the year 2013. The script will output a static JSON (JavaScript Object Notation, a lightweight data-interchange format) file containing metrics for front-end visualization.

2. Authorized Data Sources

According to the Google Earth Engine (GEE) Data Catalog and standard remote sensing practices, the engineer must utilize the following datasets:

- **Thermal Data:** USGS Landsat 8 Level 2, Collection 2, Tier 1.
 - *Source Justification:* According to the United States Geological Survey (USGS), Level-2 data provides analysis-ready Land Surface Temperature (LST), eliminating the need to manually calculate temperature from raw satellite thermal bands.
- **Land Use Land Cover (LULC):** MODIS Land Cover Type Yearly (MCD12Q1) or Copernicus Global Land Service.
 - *Source Justification:* These datasets provide continuous yearly updates from 2013 onwards, necessary for dynamic masking.
- **Spatial Boundaries (Wards):** OpenStreetMap (OSM) via API or static GeoJSON (a format for encoding a variety of geographic data structures) files sourced from DataMeet India (an open-data community providing accurate Indian administrative boundaries).

3. Processing Pipeline (Methodology)

Step 3.1: Spatial Boundary Definition

1. **Ingest Ward Boundaries:** Load the city's ward boundaries as a FeatureCollection (a GEE data type representing a list of geographic shapes). Example: Uploading the GeoJSON of Mumbai's 24 municipal wards.
2. **Define City Perimeter:** Dissolve (merge) all inner ward boundaries to create a single exterior polygon representing the official city limit.

Step 3.2: Dynamic Rural Reference (The Buffer)

1. **Generate Buffer:** Programmatically draw a 10-kilometer buffer (a digital ring expanding outward) around the dissolved city perimeter.
2. **Exclude Urban Sprawl:** Subtract the city polygon from the buffer so only the outer ring

remains.

3. **Apply Annual LULC Mask:** For the specific year being calculated (e.g., 2018), load that year's LULC map. Filter the 10km buffer zone to strictly keep pixels classified as "Vegetation," "Forest," or "Cropland."
4. **Exclude Water and Elevation:** According to standard thermal remote sensing methodology, water bodies and high-elevation areas skew baseline temperatures. Apply a mask to remove any pixels classified as "Water" in the LULC map, and use a Digital Elevation Model (DEM) dataset to remove pixels with an elevation significantly higher than the city's average.
5. **Calculate Baseline:** The median temperature of the remaining valid pixels in this ring becomes the $LST_{\text{rural_baseline}}$ for that specific month.

Step 3.3: Data Ingestion & Pre-processing (Cloud Masking)

1. **Filter by Date and Bounds:** Query the Landsat 8 Level 2 ImageCollection for the specific 30-day target period, filtered by the city's bounding box.
2. **Apply Cloud Mask:** According to USGS guidelines for Landsat 8, use the QA_PIXEL band (Quality Assessment band, which contains data indicating the presence of clouds or cloud shadows) to mask out (delete) obstructed pixels from every image in the 30-day collection.
3. **Scale Factor:** Multiply the thermal band (ST_B10) by the USGS-defined scale factor (0.00341802) and add the offset (149.0) to convert the raw digital numbers to Kelvin, then subtract 273.15 to convert to Celsius.
4. **Temporal Compositing:** Reduce the image collection using a median reducer. This creates a single, cloud-free composite image representing the typical ground temperature for that month.

Step 3.4: SUHI Calculation

For each individual ward, the script must calculate the median LST of all pixels falling within that ward's boundaries ($LST_{\text{ward_median}}$).

The script will then execute the following calculation to determine the heat island intensity for that ward:

$$SUHI_{\text{ward}} = LST_{\text{ward_median}} - LST_{\text{rural_baseline}}$$

Note: A positive $SUHI$ value indicates the ward is hotter than the rural baseline (experiencing the heat island effect). A negative value indicates it is cooler.

Step 3.5: Aggregation and Output

1. **Format the Data:** Structure the calculated data into a nested dictionary.
2. **Calculate Leaderboard Metrics:**
 - * Calculate Month-over-Month (MoM) change.
 - o Calculate Year-over-Year (YoY) change for the same month to account for Indian seasonal variations (e.g., comparing May 2025 to May 2024, as comparing May to the monsoon month of July would be scientifically invalid).

3. **Export:** Write the output to a structured data.json file.

Example Output Structure

JSON

```
{
  "city": "Bengaluru",
  "month": "2026-04",
  "rural_baseline_celsius": 28.5,
  "wards": [
    {
      "ward_name": "Koramangala",
      "median_1st": 32.1,
      "suhi_score": 3.6,
      "mom_change": 0.4,
      "yoy_change": -0.2
    }
  ]
}
```

4. Execution Constraints

- **Compute Limits:** Because GEE limits processing time for free-tier accounts, the script should process historical data iteratively (e.g., one year per script execution) and append it to the database, rather than attempting to calculate 2013-present in a single runtime block.
- **Automation:** The script must be fully executable via GitHub Actions. GEE authentication must be handled securely using a Service Account JSON key stored in GitHub Secrets.